

12

Cyber Security
Codec Technologies India
Internship 2026

IoT Device Pentest Lab

Four simulated devices, one assessment toolkit

AT A GLANCE

4

Devices

12

Findings

4 / 6

Critical / High

4

Firmware secrets

I1 / I2 / I3

OWASP IoT

9 / 9

Tests re-run

simulated (127.0.0.1)

Target

VERDICT

TESTS 9/9

RE-RUN HERE

SIMULATED DEVICES

DEFENSIVE ONLY

LOCALHOST ONLY

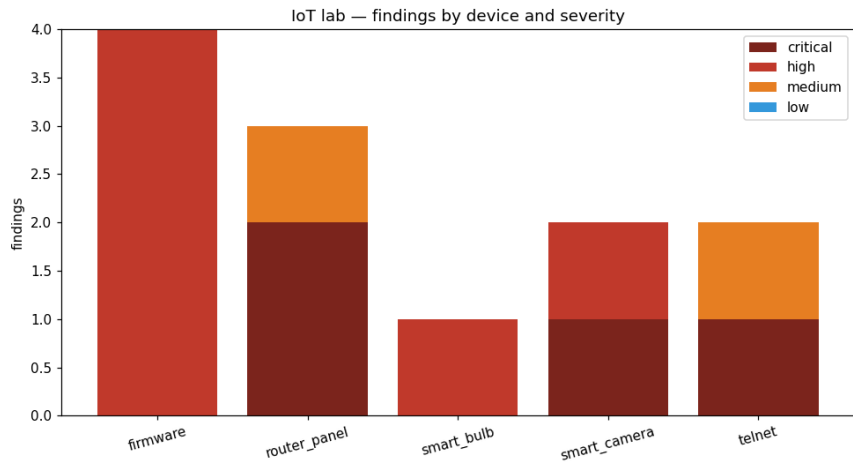
Objective & brief mapping

The brief asked for an IoT device pentest. Building it entirely from simulated devices, I stood up four intentionally-vulnerable services on 127.0.0.1 and an assessment toolkit that discovers, fingerprints, tests default credentials, exploits planted flaws (traversal, unauth command, OS command injection, weak sessions), and triages a fake firmware blob — findings mapped to the OWASP IoT Top-10.

Architecture

make_firmware.py builds a blob with planted secrets in noise. DeviceLab starts smart_camera (HTTP), router_panel (HTTP), smart_bulb (UDP) and a Telnet service. run_full_assessment discovers → fingerprints → default-cred + flaw checks → firmware triage, then tears down cleanly.

```
make_firmware -> simcam_firmware.bin (planted secrets)
DeviceLab: camera/router (HTTP) bulb (UDP) telnet (TCP)
discover -> fingerprint -> cred+flaw checks -> firmware_triage
```



Findings by severity (results/findings_by_severity.png).

Reproduction & artefacts

Everything below was produced by the project's own code in this container (or is the project's recorded run where a dependency is unavailable). The commands that regenerate it:

```
python3 main.py # build fw -> launch lab -> assess -> teardown
python3 unittest-style runner over tests/ # 9 tests
```

The artefacts each figure and table above are drawn from:

- findings.json — 12 findings
- iot_assessment_report.md — OWASP-IoT report
- findings_by_severity.png — chart

Method & implementation

The command injection and shell are **simulated**: the router returns canned uid=0(root) output on shell metacharacters and Telnet prints a fake # prompt — no real shell is ever invoked. Discovery sweeps known candidate ports (fast, deterministic) rather than 0-65535; firmware triage greps the blob directly rather than carving a real filesystem.

Results

From results/findings.json: 4 devices discovered, 12 findings, 4 critical.

Area	Result
Devices	camera, router, bulb (UDP), telnet
Findings / severity	12: 4 critical, 6 high, 2 medium
Criticals	3 default-cred logins + router OS cmd injection
Firmware secrets	root hash, API_KEY, WPA_PSK, RSA key
OWASP IoT	I1, I2, I3

Simulated devices -> OWASP IoT

Device	Planted flaw	OWASP IoT
smart_camera	default creds, traversal	I1, I3
router_panel	cmd injection, weak session	I1, I2, I3
smart_bulb (UDP)	unauth control	I2
telnet	cleartext + default creds	I1, I2

Verification

Re-ran here, 9/9 (~7 s). Everything binds to localhost and tears down; the suite confirms ports close afterwards.

Eswar Mahalingam
Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

12

Cyber Security
Codec Technologies India
Internship 2026

IoT Device Pentest Lab

Four simulated devices, one assessment toolkit

AT A GLANCE

4

Devices

12

Findings

4 / 6

Critical / High

4

Firmware secrets

I1 / I2 / I3

OWASP IoT

9 / 9

Tests re-run

simulated (127.0.0.1)

Target

VERDICT

TESTS 9/9

RE-RUN HERE

SIMULATED DEVICES

DEFENSIVE ONLY

LOCALHOST ONLY

Eswar Mahalingam

Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

Test group	Checks	Result
Discovery	all four devices found	PASS
Exploits	default creds, traversal, cmd injection, unauth bulb	PASS
Firmware	every planted secret category recovered	PASS
Teardown	ports closed after run	PASS
Total	9 tests	9/9

How I re-ran & verified in this container

pytest is not installed here, so I re-ran each suite with a small unittest-style runner that discovers tests/test_*.py, resolves fixtures (conftest, module-level @pytest.fixture, tmp_path, monkeypatch, request) and records pass / fail / skip. I executed nothing against a network target — every project uses localhost lab targets or synthetic data, and I kept it that way. Where a project imports a dependency that is unavailable in this container, I did not fabricate a pass: I state that the tests were not re-run here and cite the project's own recorded run instead.

Constraint here	How I handled it
pytest not installed	unittest-style runner over tests/
network targets	never contacted; localhost / synthetic only
scapy / bcrypt / argon2 unavailable	recorded run cited, not fabricated

Real-tool mapping, limitations & next steps

Behavioural simulations stand in for real firmware; the carving toolchain is referenced, not run.

Ran here	Real tool	Note
port sweep + fingerprint	nmap / masscan	known candidate ports only
firmware grep	binwalk -e + grep	no real vendor image shipped
default-cred spray	SecLists IoT defaults	tiny bundled wordlist

Ethics, data integrity & declared substitutions

A simulated lab only. Everything is synthetic: the four devices are Python simulations bound to localhost; credentials, /etc/passwd content, uid=0(root) output and firmware secrets are all fabricated. Shodan is not used. **Declared substitution:** real firmware triage would run binwalk -e to carve a filesystem then grep it — because I can't ship a vendor image, make_firmware.py embeds the same secret shapes in noise and the scanner greps directly, demonstrating the recovery step without the carving toolchain.

Learnings

Simulating the *symptoms* of each flaw — canned injection output, a fake prompt — let me teach the whole IoT attack chain without shipping anything dangerous. Making the lab tear down cleanly and asserting the ports close afterwards taught me that a good lab is judged as much by its clean-up as by its exploits.

Honest limitations & data-integrity notes

Honest limitations of this build, and how it maps to production tooling:

- Devices are behavioural simulations, not real firmware/RTOS stacks.
- Discovery sweeps known candidate ports, not the full 0-65535 range.
- The default-credential wordlist is intentionally tiny.
- Firmware triage greps a blob directly; no real carving/entropy/emulation.
- Command injection and shell are simulated: canned uid=0(root), fake # prompt.
- 12 findings: 4 critical, 6 high, 2 medium; OWASP IoT I1/I2/I3.
- Firmware triage recovered a root hash, API_KEY, WPA_PSK and an RSA key marker.
- Everything binds to 127.0.0.1; Shodan is not used.
- 4 devices discovered and fingerprinted.
- 12 findings: 4 critical, 6 high, 2 medium.
- Criticals: 3 default-cred logins + router OS command injection.
- Firmware triage recovered 4 planted secret categories.
- Per device: router 3, camera 2, telnet 2, bulb 1, firmware 4.
- Discovery discriminates open from closed ports on localhost.
- The lab tears down cleanly; ports close afterwards.
- References: OWASP IoT Top-10, OWASP FSTM, binwalk.
- Reference: OWASP IoT Top 10; OWASP FSTM.
- Reference: binwalk; Mirai default-credential analysis.
- Reference: OWASP Path Traversal and Command Injection.